TCP Client Experiment

1. TCP Client Overview

TCP (Transmission Control Protocol) is a connection-oriented reliable transport protocol that guarantees in-order arrival and no packet loss. In this experiment the ESP32-S3 acts as a TCP client, connecting to the network debugging assistant (in TCP Server mode) on the PC to achieve bidirectional send/receive: the client can receive commands from the PC and reply (such as `TIME` / `INFO` / `PING`), while periodically sending heartbeat packets to maintain the connection.

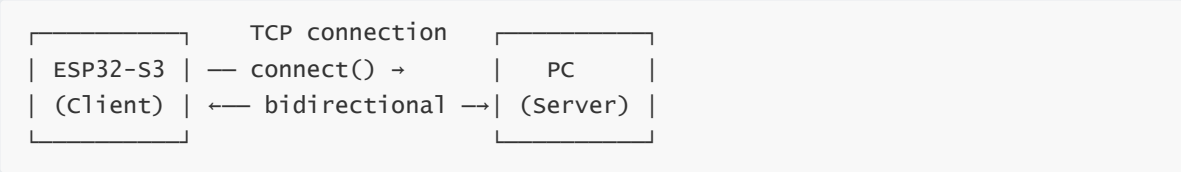
Typical application scenarios:

Application Type	Example
Remote control	Phone/PC sends commands to control ESP32 peripherals
Data reporting	Sensors push data to a server periodically
File transfer	Firmware OTA upgrade
Chat communication	Simple TCP chat terminal

> The PC needs to run a TCP Server (such as the NetAssist network debugging assistant) listening on a specified port, waiting for the ESP32 to connect.

2. Core Concepts

2.1 TCP Client Communication Model



- TCP is connection-oriented; you must `connect()` to establish a connection before communicating
- After the connection is established, both sides can `send()` / `recv()`
- If either side calls `close()` or loses the network, the connection is broken and reconnection is needed

2.2 Configuration Parameters

Parameter	Description	Value Used in This Experiment
WIFI_SSID / WIFI_PASS	WiFi connection	`"Yahboom2"` / `"yahboom890729"`
SERVER_IP	PC IP (server address)	`192.168.11.183`
SERVER_PORT	Port the PC is listening on	`8080`
HEARTBEAT_INTERVAL	Heartbeat interval	10s
RECONNECT_INTERVAL	Disconnect reconnect interval	5s

2.3 Core API

```python
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  # TCP socket
sock.connect(("192.168.11.183", 8080))           # connect to server
sock.sendall(b"hello")                           # send data
data = sock.recv(512)                            # receive (up to 512 bytes)
sock.close()                                     # close
```

2.4 Command Set

The PC sends the following text commands through the network assistant; the ESP32 receives them and dispatches them in `process_command()`:

Command	Function	Reply Content
`PING`	Connectivity check	`PONG`
`TIME`	Query runtime status	Runtime (s) + free memory (KB)
`INFO`	Query device info	Free memory + total memory + usage + Flash capacity
Other	Echo	`Received: {original message}`

> Command matching is case-sensitive; `ping` is not recognized as PING. Commands do not contain leading/trailing spaces (`recv` performs a `strip()`).

---

3. Hardware Dependencies

The ESP32-S3 has built-in WiFi. A PC running the network debugging assistant (in TCP Server mode, listening on `SERVER_PORT`) is needed, on the same LAN as the ESP32-S3.

---

4. Project Architecture and Flow

```
Script execution (single-file .py)
  |
  |- wifi_connect(): connect to WiFi (STA mode)
  |- create_socket() / connect_server(): create socket -> connect to PC
  |
  |- while True:
       |- recv_msg(): non-blocking reception of PC commands
       |- process_command(): command dispatch (TIME/INFO/PING)
       |- send_msg(): reply with processing results
       |- heartbeat: send HEARTBEAT every 10s
```

---

5. Source Code Explained

Full source code: `Source Code -> MicroPython -> 4.Network Protocols -> TCP_Client -> TCP_Client.py`

5.1 Configuration Area

The server IP and port, heartbeat interval, and reconnect interval can all be modified as needed.

```python
SERVER_IP    = "192.168.11.183"   # PC IP
SERVER_PORT = 8080               # PC listen port
HEARTBEAT_INTERVAL = 10          # heartbeat interval (s)
RECONNECT_INTERVAL = 5           # reconnect interval (s)
```

5.2 Creating the Socket and Connecting

`socket.SOCK_STREAM` = TCP; `settimeout(5)` prevents `recv` from blocking indefinitely.

```
def create_socket():
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  # TCP socket
    sock.settimeout(5)
    return sock

def connect_server(sock, ip, port):
    sock.connect((ip, port))                        # connect
    print(f"[TCP] Connected to server {ip}:{port}")
```

5.3 Sending and Receiving Data

`sendall` ensures the data is sent completely; an empty byte string returned by `recv` means the peer has closed the connection.

```
def send_msg(sock, msg):
    data = msg.encode("utf-8") if isinstance(msg, str) else msg
    sock.sendall(data)                              # send all

def recv_msg(sock, buf_size=512):
    data = sock.recv(buf_size)                      # blocking receive
    if data:
        return data.decode("utf-8")
    else:
        return None  # connection closed
```

5.4 Command Dispatch

Supports `TIME` (runtime status), `INFO` (device info), and `PING` (replies PONG); all other messages are echoed as-is.

```
def process_command(msg):
    if msg == "TIME":
        return f"uptime: {uptime:.0f}s, Free memory: {free / 1024:.0f} KB"
    elif msg == "INFO":
        return f"ESP32-S3 Client, Free memory: {gc.mem_free() / 1024:.0f} KB"
    elif msg == "PING":
        return "PONG"
    else:
        return f"Received: {msg}"                       # echo
```

5.5 Main Loop

Auto-reconnect on disconnect + receive command processing + send heartbeat every 10s.

```
while True:
    if sock is None:                                # reconnect if needed
        sock = create_socket()
        if not connect_server(sock, SERVER_IP, SERVER_PORT):
            sock = None; time.sleep(RECONNECT_INTERVAL)
            continue
```

```
    msg = recv_msg(sock)
    if msg:
        reply = process_command(msg)
        if reply:
            send_msg(sock, reply)

    if time.time() - last_heartbeat > HEARTBEAT_INTERVAL:  # heartbeat
        send_msg(sock, "HEARTBEAT")
        last_heartbeat = time.time()
```

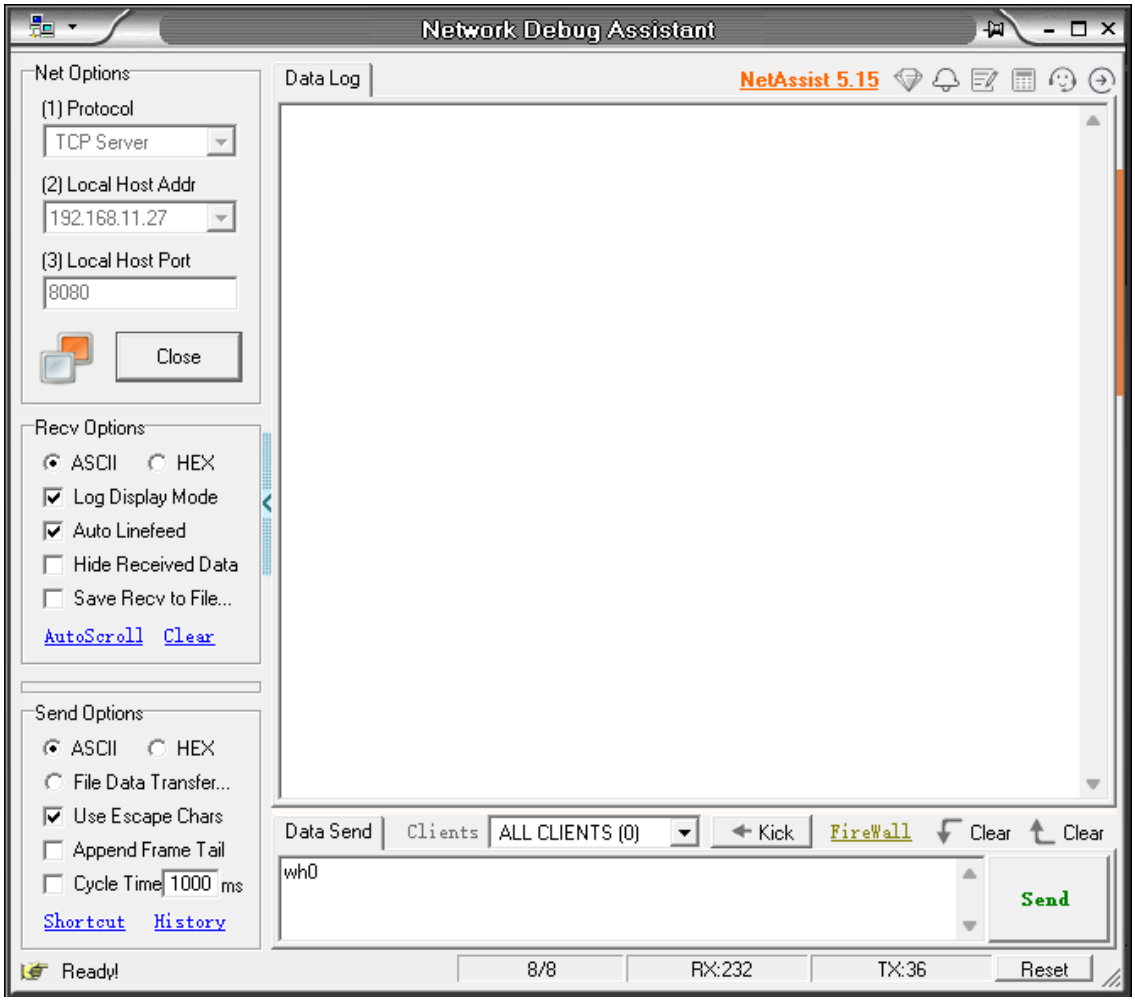
6. Operating Procedures

6.1 Environment Preparation

Thonny IDE steps: See `MicroPython -> 1. Before Development -> 3.Thonny IDE`.

1. Open the network debugging assistant 

on the PC, select **TCP Server** mode, and set the listen port to `8080`



2. Confirm the PC IP (`ipconfig`) and modify `SERVER_IP` in the code
3. Confirm the ESP32-S3 and the PC are on the same WiFi

6.2 Flashing and Running

1. Open `TCP_Client.py` in Thonny IDE, modify `WIFI_SSID`, `WIFI_PASS`, `SERVER_IP`, and `SERVER_PORT`

```
# ==================== 配置区 / Config ====================
WIFI_SSID    = "Yahboom2"             # WiFi名称 / WiFi SSID
WIFI_PASS    = "yahboom890729"        # WiFi密码 / WiFi password

SERVER_IP    = "192.168.11.27"       # 服务器 IP / server IP
SERVER_PORT = 8080                    # 服务器端口 / server port

RECONNECT_INTERVAL = 5               # 断线重连间隔(s) / reconnect interval
HEARTBEAT_INTERVAL = 10              # 心跳间隔(s) / heartbeat interval
# ========================================================
```

2. Click Run

6.3 Serial Output Example

```
===== ESP32-S3 TCP Client =====
[WiFi] Connecting to 'Yahboom2' ...
[WiFi] Connected! IP: 192.168.11.229
[TCP] Connected to server 192.168.11.27:8080
[Sent] HEARTBEAT
[Recv] TIME
[Sent] ESP32-S3 status
  uptime: 3320s
  Free memory: 8119 KB
[Sent] HEARTBEAT
[Recv] INFO
[Sent] ESP32-S3 Client
  Free memory: 8117 KB
  Total memory:   8127 KB
  utilization:   0.1%
  Flash:    16 MB
[Recv] PING
[Sent] PONG
[Sent] HEARTBEAT
```

```
===== ESP32-S3 TCP Client =====
[WiFi] Connecting to 'Yahboom2' ...
[WiFi] Connected! IP: 192.168.11.229
[TCP] Connected to server 192.168.11.27:8080
[Sent] HEARTBEAT
[Recv] TIME
[Sent] ESP32-S3 status
  uptime: 3320s
  Free memory: 8119 KB
[Sent] HEARTBEAT
[Recv] INFO
[Sent] ESP32-S3 Client
  Free memory: 8117 KB
  Total memory:   8127 KB
  utilization:   0.1%
  Flash:    16 MB
[Recv] PING
[Sent] PONG
[Sent] HEARTBEAT
```

6.4 Verification Methods

Operation	Expected Result
PC sends `PING`	ESP32 replies `PONG`
PC sends `TIME`	ESP32 replies runtime + free memory
PC sends `INFO`	ESP32 replies memory + Flash info
PC closes the Server	ESP32 reconnects every 5s
Wait 10s	ESP32 automatically sends HEARTBEAT

---

7. Common Problems

Symptom	Cause	Solution
Connection fails `ECONNREFUSED`	PC Server not started or wrong port	Confirm the network assistant has started a TCP Server with port=8080
Connection timeout	IP not on the same subnet or firewall blocking	`ping` the PC IP to test reachability, disable the firewall
Cannot receive PC messages	`recv()` blocking causes heartbeat to stop	Already using `settimeout(0.5)` non-blocking polling
Garbled Chinese characters	Inconsistent encoding	Use UTF-8 encoding/decoding uniformly